



Lab1实验报告

一、实验思路

主要任务包括：完成 SysY 语言的词法分析和语法分析，构建抽象语法树，基于 Flex 和 Bison 框架实现

1. 词法分析(lexer.l)

- 实现的token类型包括：关键字、运算符、分隔符、标识符、常量、注释
- 实现要点
 - 正则表达式：定义标识符、数字常量的模式
 - 关键字优先：先匹配关键字，再匹配标识符
 - 多进制整数支持：通过正则表达式 `0[0-7]+` 和 `0[xX][0-9a-fA-F]+` 分别匹配八进制和十六进制整数，使用 `strtol` 进行进制转换
 - 忽略注释和空白：不返回token

2. 语法分析(parser.y)

- 采用自底向上的 LALR(1) 解析：使用Bison生成移进-归约解析器
- 消除歧义的设计：将 `FuncType` 直接内联到 `FuncDef` 规则中，避免 `int IDENT` 既能归约为变量声明又能归约为函数定义带来的冲突

3. AST构建

- 每种语法结构对应一个AST节点子类，继承自基类 `Node`
- 使用 `std::shared_ptr` 管理子节点，通过 `shared_cast` 辅助函数在 `Bison union` 指针和智能指针之间进行切换

二、难点与亮点

难点一：shift/reduce冲突问题

- `int main(){...}` 被错误解析为变量声明，bison在看到 `int IDENT` 后遇到 `(` 时选择 shift(继续作为变量解析)而非 reduce(将int main归约为函数定义)

- 解决思路：分析发现冲突根源

是 `VarDecl: "int" VarDefs ";"` 与 `FuncDef: FuncType IDENT "(" ")" Block` 共享前缀 `int IDENT`，通过将 `FuncType` 非终结符消除，让 `int` 和 `void` 直接出现在 `FuncDef` 规则中，使 Bison 在遇到 `(` 时可以区分两种情况。

难点二：多维数组的声明与访问

- `int a[2][2] = {1, 2, 3, 4};` 和 `array[0][0][0]` 解析失败
- 解决方法
 - 不同维度的数组使用多条产生式去覆盖
 - 在 `LVal` 节点中添加 `index`，`index2`，`index3` 三个字段分别记录各个维度索引的表达式
 - 同时还要解决数组的初始化问题，引入了 `InitVal` 和 `InitVals` 规则支持 `{}` 初始化列表

难点三：函数形参数组

- 问题：我们在解析测试函数(`int f(int array[][2][2])`)
- 解决思路：在 `FuncParam` 中添加了 `is_array`，`array_second_dim` 和 `array_third_dim` 三个字段，通过 `int IDENT "[" "]" "[" INTCONST "]" "[" INTCONST "]"` 产生式匹配这种省略第一维的数组形参语法
- 这里我们灵活运用了 Bison 语义值：通过 `shared_cast` 模板函数和 `take_string` 辅助函数，解决了 `Bison union` 无法直接使用 `std::shared_ptr` 的问题，实现了裸指针到智能指针的安全转换

三、心得体会

课堂上学习的词法分析、语法分析的理论知识都只是基础，实际动手才会发现，一个最简单的编译器需要注意的细节会更多，包括语法考虑的全面性、flex 规则的顺序、bison 冲突的处理策略等等，一个小的失误都会使实际的编译器无法正常工作。同时我还意识到我们要擅长使用工具输出，比如 bison 的 `-v` 参数生成的 `.output` 文件对非冲突产生的具体原因非常有帮助，能够清除看到每个状态的冲突项。整个 lab1 实验的增量式调试的过程也让我感受到了工程代码的先让基本框架 work 起来，再逐步去修改比一次性写完所有的要高效的多(本人就是从 26 分一路改到 43 分)

```
Error: 26/43 tests passed.  
2026-04-04 18:53:10 UTC+08:00  
(user) (host) ...
```

```
All tests passed!  
  
Error: reports/lab1.pdf not found.  
Test score: 100.00  
2026-04-05 14:59:20 UTC+08:00
```

AI使用截图



杨思行
04/04 18:08

```
```yacc  
UnaryExp能解释一下这个变量的含义是什么吗
```
```





MiniMax-M2.7
04/04 18:08

⌕ THINKING

用户在问 UnaryExp 的含义。根据 SysY 语法规则，UnaryExp 是 unary expression（一元表达式）的缩写，表示一元运算符（正号、负号、逻辑非）组成的表达式。

UnaryExp 解释

UnaryExp 是 Unary Expression（一元表达式）的缩写。

根据 SysY 语法规则：

Code

```
UnaryExp ::= PrimaryExp  
          | IDENT "(" [FuncRParams] ")"  
          | UnaryOp UnaryExp;
```

复制

组成形式

1. PrimaryExp - 基本表达式（变量、常量、括号表达式）

3. UnaryOp UnaryExp - 一元运算符 + 一元表达式

示例

Code 复制

```
-a      // UnaryOp::Neg + LVal(a)
+b      // UnaryOp::Pos + LVal(b)
!x      // UnaryOp::Lnot + LVal(x)
!0      // UnaryOp::Lnot + IntConst(0)
-foo()  // UnaryOp::Neg + FuncCall(foo)
```

优先级

在 SysY 表达式优先级中：

- ``UnaryExp`` 优先级高于 ``MulExp``
- 一元运算符优先级最高（仅次于括号）

所以 ``UnaryExp`` 就是包含一元运算符（``+``、``-``、``!``）的表达式。

13.8s 复制



杨思行
04/04 18:11

那为什么乘法表达式要使用UnaryExp



MiniMax-M2.7
04/04 18:11

THINKING

用户问了一个很好的问题：为什么乘法表达式(MulExp)要使用UnaryExp作为操作数？
这是因为SysY的语法定义遵循以下规则：

Code 复制

展开思考

这是一个很好的问题！

原因：表达式的优先级

SysY 语法中，一元运算符（``+``、``-``、``!``）的优先级高于乘除模运算。

优先级层次

Code 复制

```
UnaryOp (!, +, -)  ← 最高
MulOp  (*, /, %)
AddOp  (+, -)
RelOp  (<, >, <=, >=)
EqOp   (==, !=)
LAndOp (&&)
LOrOp  (||)       ← 最低
```

示例解析

``!a * b`` 应该如何解析？

按照语法：



按照语法：

Code 复制

```
UnaryExp ::= UnaryOp UnaryExp | PrimaryExp;
MulExp   ::= UnaryExp | MulExp "*" UnaryExp;
```

解析过程：

Code 复制

```
!a * b
= UnaryExp(!, a) * UnaryExp(b)    ← !a 是 UnaryExp
= MulExp(UnaryExp(!, a)) * UnaryExp(b)
= MulExp(!a) * MulExp(b)
```

如果 `MulExp` 直接用 `MulExp` 作为操作数，就无法正确处理这种情况。

对比

| 表达式 | 错误解析 | 正确解析 |
|-----------------------|---------------------------|---------------------------|
| <code>`a * b`</code> | <code>`a * b`</code> (一样) | <code>`a * b`</code> |
| <code>`!a * b`</code> | <code>`!(a * b)`</code> ❌ | <code>`(!a) * b`</code> ✅ |
| <code>`-a + b`</code> | <code>`-(a + b)`</code> ❌ | <code>`(-a) + b`</code> ✅ |

↓

总结

`MulExp` 使用 `UnaryExp` 而不是 `PrimaryExp`，是为了确保一元运算符的优先级高于乘除模。这是 SysY 语法规范的要求。

20.4s 复制 引用



杨思行
04/04 18:14

\$\$ \$1这些符号是什么意思

复制



MiniMax-M2.7
04/04 18:14

🔄 THINKING

用户询问 parser.y 中 `$$` 和 `$1` 这些符号的含义。这些是 Bison 语法中的特殊变量。

Bison 中的 `$$` 和 `$1` 含义

在 Bison 语法中，这些是位置标记符：

| 符号 | 含义 |
|-------------------|---------------|
| <code>\$\$</code> | 规则左侧的语义值（返回值） |
| <code>\$1</code> | 右侧第 1 个符号的语义值 |
| <code>\$2</code> | 右侧第 2 个符号的语义值 |
| <code>\$3</code> | 右侧第 3 个符号的语义值 |

↓

... 以此类推

示例解释

Code

复制

```
AddExp : MulExp { $$ = $1; }  
  | AddExp "+" MulExp { $$ = new BinaryExp(BinaryOp::Add, NodePtr($1), NodePtr($3)); }
```

规则 1: `AddExp : MulExp`

- `\$\$` = `AddExp` 的返回值
- `\$1` = `MulExp` 的值
- 语义: 直接返回 `MulExp` 的值

规则 2: `AddExp : AddExp "+" MulExp`

- `\$\$` = `AddExp` 的返回值
- `\$1` = 第一个 `AddExp` 的值
- `\$2` = `+` (忽略, 通常为空白)
- `\$3` = `MulExp` 的值
- 语义: 创建一个 `BinaryExp` 节点, 左操作数是 `\$1`, 右操作数是 `\$3`



杨思行

04/04 18:29

stmt中的问题不是因为我们没有定义if这个关键字吗



MiniMax-M2.7

04/04 18:29

THINKING

用户提出了一个问题, 说 Stmt 中的问题是不是因为没有定义 `if` 这个关键字。

让我检查一下 lexer.l 和 parser.y 中的 token 定义:

从 system-reminder 中可以看到 lexer.l 已经定义了:

展开思考

你说得对! 我发现问题了:

问题: lexer.l 返回了 `IF`, 但 parser.y 没有声明它

lexer.l (第48行)

Code

复制

```
"if"          { return IF; } // ✓ 返回了 IF
```

parser.y

Code

复制

```
%token ELSE "else"    // ✓ 有 ELSE  
%token WHILE "while"  // ✓ 有 WHILE
```



Code

复制

```
%token ELSE "else"    // ✓ 有 ELSE
%token WHILE "while"  // ✓ 有 WHILE
// 缺少: %token IF "if"
```

修正

在 parser.y 中添加 `IF` token 定义:

Code

复制

```
%token IF "if"
%token ELSE "else"
%token WHILE "while"
%token VOID "void"
```

同时 `FuncDef` 规则中使用的是 `int` 而不是 `INT`, 所以 `FuncType` 规则应该改为:

Code

复制

```
FuncType : "int" { $$ = BasicType::Int; }
         | "void" { $$ = BasicType::Void; }
         ;
```

修改后再次编译测试。

18.7s

复制

↓

6:45:34 PM You

我们现在已经根据/Users/yangsmac/Desktop/compiler/sp26/docs/lab/lab1.md完成了代码的编写, 但是现在我们编译存在上述问题, (base) 🐉 base ~/Desktop/compiler/cp-3230104703 lab1 ± make clean && make

```
rm -f src/lexer/lexer.yy.cc src/parser/parser.tab.cc src/parser/parser.tab.hh rm -f src/ast/tree.o src/main.o src/lexer/lexer.yy.o src/parser/parser.tab.o rm -f src/ast/tree.d src/main.d src/lexer/lexer.yy.d src/parser/parser.tab.d rm -f compiler bison -d -o src/parser/parser.tab.cc src/parser/parser.y src/parser/parser.y: conflicts: 1 shift/reduce flex -o src/lexer/lexer.yy.cc src/lexer/lexer.l g++ -std=c++17 -g -Wall -MMD -Isrc -c src/lexer/lexer.yy.cc -o src/lexer/lexer.yy.o In file included from src/lexer/lexer.l:7: src/ast/tree.hpp:162:2: warning: Have not support params yet [-W#warnings] 162 | #warning Have not
```

Thinking

The user is having a compilation error. The error shows that `BinaryOp::And` and...

Grep

INPUT

>  Grep

 Thinking

Let me search for BinaryOp more broadly to find its definition.

>  Grep

 Thinking

I need to find where 'BinaryOp' is defined. Let me search for its definition.

>  Grep

 Thinking

Let me read the beginning of the tree.hpp file to understand the structure and f...

>  Read tree.hpp

 Thinking

I need to find where BinaryOp is defined. It's likely in common.hpp. Let me chec...

>  Read common.hpp

 Thinking

Now I see the issue. In 'common.hpp' line 24, the 'BinaryOp' enum has 'LAnd' and...

 MiniMax-M2.7 6:46:44 PM

找到问题了。在 'common.hpp' 中定义的二元运算符枚举是 'LAnd' 和 'LOr'，但 'parser.y' 中使用的是 'And' 和 'Or'。

让我查看并修复 parser.y:

让我查看并修复 parser.y:

>  Read parser.y

 Thinking

Now I can fix the issue. I need to change `BinaryOp::And` to `BinaryOp::LAnd` an...

>  Edit

 Thinking

The user wants to use `And` and `Or` instead of `LAnd` and `LOr`. So I need to u...

 MiniMax-M2.7 6:47:53 PM

明白, 需要在 `common.hpp` 中将 `LAnd` 和 `LOr` 改为 `And` 和 `Or`:

>  Edit common.hpp

 Thinking

Now I need to also update the switch statement for op_to_string to use `And` and...

>  Edit common.hpp

 Thinking

Now let me rebuild to verify the fix works.

 MiniMax-M2.7 6:48:27 PM

现在重新编译验证:

>  Bash

> >_ Bash

Thinking

The compilation succeeded. There are still some warnings about: 1. "Have not sup...

MiniMax-M2.7 6:49:02 PM

编译成功了! ``BinaryOp::And`` 和 ``BinaryOp::Or`` 的问题已修复。

编译还有一些警告:

1. ``parser.y: conflicts: 1 shift/reduce`` - 有一个语法冲突
2. `tree.hpp` 中的两个 ``#warning`` 提示尚未支持的功能

但主要错误已解决, ``compiler`` 可执行文件已生成。